

Compression d'images à l'aide des k moyennes

L'algorithme des k-moyennes peut-être utilisé pour compresser des images. C'est tout l'objet de ce TP.

L'idée est de regrouper ensemble des pixels visuellement proches, et de leur assigner la même valeur. Cela permet de réduire le nombre de couleurs de l'image.

A Préambule

A.1 Portable Network Graphics

Le format d'image que l'on utilisera est **Portable Network Graphics**, plus connu sous le nom de **PNG**. C'est un format compressé : la suite de pixels est compressée à l'aide de l'algorithme Deflate, qui est un mélange de l'algorithme de Huffman et de LZ77 (compression par motif, comme LZW).

1. Rappeler rapidement le principe des algorithmes de Huffman et de LZW. Si vous n'y parvenez pas, pensez à planifier une séance de révision.

Grâce à ces compressions, moins il y a de couleurs dans l'image, et plus son PNG sera léger ! Réduire le nombre de couleurs d'une image permet donc de gagner de l'espace de stockage.

Pour ce TP, vous ne manipulerez pas les PNG (et la compression/décompression directement). Tout cela a été précodé. Vous aurez uniquement à implémenter l'algorithme des k moyennes.

Il vous faut cependant savoir que :

- Un fichier PNG est constitué de meta-données, suivies de la compression de la suite de pixels. La partie pré-codée de ce TP n'essaye pas de conserver les méta-données : aussi, tout png produit durant ce TP gagne (par rapport à l'original) quelques kilo-octets « en trichant ».
- Il existe différentes façons de coder des pixels. Dans ce TP, le code donné impose le format RGBA. C'est à dire qu'un pixel est composé de :
 - Un champ **r** : c'est l'intensité de son rouge (red).
 - Un champ **g** : c'est l'intensité de son vert (green).
 - Un champ **b** : c'est l'intensité de son bleu (blue).
 - Un champ **a** : c'est son opacité (à quel point le pixel est-il non-transparent ?). On nomme cela un **canal α** .

Les trois couleurs Rouge Vert Bleu, en se mélangeant, simulent tout le spectre lumineux. Le canal α permet d'ajuster la luminosité des pixels, et permet d'avoir par exemple un fond transparent sur l'image.

A.2 Conséquences

Dans ce TP vous implémenterez k-moyennes pour RGBA. Cela signifie que les PNG manipulés *doivent* avoir un canal α . Le code pré-codé fournit une option `add-alpha` qui crée une copie d'un png avec canal α . S'il y a déjà un canal α , l'option ne fait rien.

Aussi, pour analyser la performance de votre algorithme, vous ferez les étapes suivantes :

- Création d'une copie de l'image d'origine en format RGBA. La copie n'a (presque) pas de métadonnées.
- Compression de la copie. La version compressée n'a (presque) pas de métadonnées : en fait elle a les mêmes que la version créée par `add-alpha`.
- Comparaison des tailles de la copie et de la version compressée.

A.3 Option de l'exécutable

L'exécutable que vous allez produire a deux modes. Nommons cet exécutable `exe`.

- `./exe add-alpha src.png dst.png` crée une copie de `src.png` qu'il nomme `dst.png`. Cette copie est garantie d'avoir un canal α , mais peut avoir perdu des métadonnées.
- `./exe compress src.png dst.png k` crée une copie de `src.png` qu'il nomme `dst.png` et qui n'a que k couleurs. Elle est garantie d'avoir un canal α , et peut avoir perdu des métadonnées.

A.4 Makefile

Un Makefile est fourni pour simplifier la compilation. Pour le lancer :

- `make` compile l'exécutable.
- `make run` compile et lance l'exécutable.

Si vous voulez ajouter des options à la compilation, modifiez la ligne 5. Si vous voulez modifier les options données à l'exécutable, modifiez la ligne 15.

Initialement, la ligne 15 demande d'utiliser `add-alpha` sur l'image `gradient.png`.

A.5 Outils

La manipulation de png a été pré-codée. Sa version compilée est `liboutils.so`. Le descriptif des fonctions est dans `outils.h`.

Vous n'avez pas besoin de lire et comprendre les fonctions `decode_png` et `encode_png` fournies dans `outils.h`.

2. Lisez la définition du type `pixel` dans `outils.h`, que l'on utilise pour manipuler un pixel au format RGBA.

B k moyennes

3. Rappeler brièvement le principe de l'algorithme des k -moyennes (auss appelé algorithme de Lloyd).

B.1 Présentation des variables

Afin de mettre en oeuvre l'algorithme des k -moyennes, nous utiliserons les variables suivantes :

- k le nombre de clusters à créer. Il est lu sur la ligne de commande.
- `img` un tableau contenant les pixels de l'image les uns après les autres. Il est déjà construit dans la fonction `main` grâce aux fonctions pré-codées (qui lisent le fichier indiqué sur la ligne de commande).
- N le nombre de pixels de l'image.
- `cluster` est un tableau qui à l'indice d'un pixel associe le numéro de son cluster. Ce tableau est donc de taille N .
- `isobar` est un tableau qui au numéro d'un cluster associe son isobarycentre. Il est donc de taille k .
- `card` est un tableau qui au numéro d'un cluster associe son cardinal. Il est donc de taille k .

On rappelle que « isobarycentre » est un terme technique qui désigne simplement la moyenne. L'isobarycentre de points, c'est la somme¹ de ces points divisée par le nombre de points.

4. (Version étoilée du TP) : Lisez la fonction `main` du fichier `kmeans.c` fourni et familiarisez-vous avec.

B.2 C'est parti

Pour toutes les fonctions, la documentation précise se trouve dans `kmeans.h`. Vous pouvez tout à fait créer vos propres fonctions auxiliaires, même si elles ne sont pas demandées par l'énoncé.

5. Complétez la fonction `sqd` qui prend en argument deux pixels et renvoie le **carré** de la distance euclidienne entre eux.
6. Complétez la fonction `trouve_cluster` qui calcule et renvoie le numéro du plus proche cluster d'un pixel donné.

1. Un point est un élément de $\mathbb{R}^d$, il est donc tout à fait rigoureux de les sommer ou de les diviser par un scalaire.

La prochaine étape consiste en l'initialisation des isobarycentres. Initialement, on prend k points aléatoires. Pour générer un entier au hasard, on utilisera la fonction `rand` (qui ne prend pas d'arguments). Pour générer un entier au hasard entre 0 (inclus) et N (exclus), on utilisera `rand() % N`.

Il peut être étonnamment délicat de tirer rapidement et **uniformément** k entiers aléatoires tous différents dans $\{0, \dots, N-1\}$. On propose d'utiliser ici le mélange de Fisher-Yates pour obtenir ces k indices aléatoires de pixels parmi les indices $\{0, \dots, N-1\}$ possibles.

L'algorithme de Fisher-Yates mélange uniformément les indices $\{0, \dots, N-1\}$ d'un tableau, c'est à dire que chaque ordonnancement des indices du tableau a la même probabilité d'être obtenue par ce mélange. Nous verrons les bases théoriques qui garantissent que ce mélange fonctionne dans un futur chapitre. Pour le moment, on admet que l'algorithme suivant est correct :

Pseudo-code

```

1  Fischer-Yates (T) :
2  Pour i allant de n-1 à 1 faire :
3      j ← entier aléatoire entre 0 et i
4      échanger T[j] et T[i]
```

En partant d'un tableau T contenant les N premiers entiers dans l'ordre croissant (pour tout $i < N$, $T[i] = i$), il suffit alors de prendre les k premières cases du tableau après mélange pour avoir sélectionné k indices aléatoires de pixels.

7. Implémentez la fonction `initialise_centres` (pour vous aider, lisez sa documentation et ses commentaires).

(Rappel / Culture G. : En fait, `rand` est une suite pseudo-aléatoire : chaque appel renvoie le terme suivant d'une suite, dont la formule est assez compliquée pour qu'il soit difficile de prédire à l'avance quel sera le prochain terme. Cependant, cette suite est entièrement déterminée par son premier terme ! Il faut l'initialiser avec `srand`, par exemple en utilisant l'heure. C'est déjà fait dans le `main`, vous n'avez pas à le faire.)

8. Nous allons maintenant programmer `maj_isobar`, qui calcule l'isobarycentre de chaque cluster. Pour le calculer, on va sommer tous les pixels du cluster, puis diviser par le cardinal du cluster.

- Sachant qu'un pixel est encodé par un `uint8_t` sur un octet (donc entre 0 et 255), pour une image de taille 1920×1080 , quelle est valeur maximale d'une coordonnée (R,G,B ou A) de la somme des pixels ? Peut-elle tenir sur un `uint8_t` ? Sur un `int` ?
- On rappelle que l'on peut convertir entre types compatibles. Écrire `(t) expr` revient à évaluer `expr` et convertir le résultat dans le type `t`.

Implémenter la fonction `maj_isobar`. Elle prend en argument le tableau `cluster` et le tableau `card` déjà calculés et mis à jour (on sait donc dans cette question à quel cluster appartient chaque pixel et combien de pixels contient chaque cluster).

Dans le cas où un des clusters serait de cardinal 0, on ne modifiera pas la valeur de son isobarycentre.

Il est temps de coder l'algorithme des k moyennes en entier.

9. Implémentez la fonction `kmeans`.

Indication : Commencez par initialiser k points par les fonctions précédentes, en pensant à créer le tableau `card` utile pour la suite, puis codez les itérations : tant que cela change quelque chose, assignez chaque point à l'isobarycentre le plus proche, puis mettez à jour les isobarycentres.

Votre fonction doit également renvoyer le nombre d'itérations nécessaires.

Quand vous testerez votre fonction, il se peut qu'elle soit lente. Vous pouvez si vous le souhaitez ajouter des affichages qui indiquent l'avancement, mais attention car les affichages prennent du temps. Retirez-les avant d'envoyer votre fichier pour le testeur !

10. Dans le TP précédent, la notion de niveau d'optimisation de gcc a été présentée. Il a notamment été expliqué qu'une opération souvent effectuée dans l'algorithme des plus proches voisins était optimisable sur des processeurs modernes. Laquelle ? Est-elle présente ici ?

On évitera d'utiliser `-O3` qui pourrait casser la manipulation des PNG. Les niveaux précédents sont sans danger par contre.

11. Amusez-vous : testez votre fonction ! Comparez les poids !